Sandya Mannarswamy

CODE SPORT

In this month's column, we continue our discussion on information retrieval.

In last month's column, we discussed algorithms for constructing an inverted index, which is the basic data structure used in all information retrieval systems. We discussed two algorithms—the Blocked Sort Based Indexing (BSBI) method and the Single Pass In-Memory (SPMI) method for index construction. Both these algorithms break up the document collection into blocks, which are processed one at a time, since it is typically not possible to hold the entire document collection in the main memory for processing, to build the index. This imposes considerable complexity unlike the normal sort/search algorithms we come across while programming, in which the complete data set can be held in the main memory. It also imposes different performance challenges.

In our day-to-day programming, we typically use algorithms which operate on data in the main memory. So we are mostly concerned with the performance impact of the CPU and memory on our code. However, in the case of index construction algorithms like BSBI or SPMI, we are concerned with a third factor—the secondary storage where we store intermediate results and read data for further processing. We need to keep the disk access speeds and disk bandwidth as potential factors when designing these algorithms.

So far, we have been discussing index construction algorithms that work on a single machine, and the only hardware constraint we have considered so far is the amount of available main memory in the computer system in which the document collection is being processed. However, consider the case of constructing an inverted index for the document collection which comprises the 'World Wide Web'. This task cannot be done on a single computer system and, hence, needs to be carried out on a very large cluster of machines. Therefore, a distributed method for 'inverted index' creation is needed, by which the index construction can be spanned across multiple nodes of the cluster, thereby utilising the CPU and physical memory available on each of these nodes. A well-known distributed computational model is the 'map reduce' paradigm, and this has been adapted to the task of distributed index creation.

I am sure that our readers are familiar with the basic idea of Map-Reduce, where a task is split into the 'map' phase and the 'reduce' phase. During the map phase, the mapper task is run on multiple nodes on the cluster, with the master node of the cluster assigning the task to each mapper node. After the map phase is completed, the reducer phase is run, on the nodes of the cluster. In the case of distributed index creation, the document collection is first split into 'blocks' by the master node, which assigns these split blocks to mapper nodes. During the mapping phase, the split block is parsed by the mapper node and, as in the case of the single node index creation algorithm, a list of *<term id, doc id>* is constructed for each split block at each mapper node.

Note that the split blocks are not assigned statically to mapper nodes. It is expected that mapper nodes can fail or become unresponsive during the course of computation. Hence, the master node assigns one split block at a time to a mapper node. Only after successfully processing the current split block, the mapper node is assigned the next block for processing. In case the mapper node becomes unresponsive after a time-out period has elapsed, the master node declares the mapper node to have failed and removes that node from the map-reduce computation. The split block which was being processed by the failed node is reassigned to an 'alive' node. As mentioned before, each mapper node produces a list of *<term id, doc id>* pairs at the end of the map phase. This is written into intermediate local files on each mapper node. These files are typically known as segment files.

During the 'reduce' phase, we would like to process the *<term id, doc id>* lists produced during the 'map' phase and construct the pos-list, which gives the final *<term id*, a list of doc IDs where the term appears> in the entire document collection. This requires that all lists corresponding to the same term ID be processed at a single node so that inter-node communication is